

Name: Shraboni Kumar

SAP ID: 500129767

Course Name: Data Structure using C

Assignment ID: Assignment-2

1.

The Advantage of Circular Queues:

The primary disadvantage of an ordinary queue implemented with an array is the "False Overflow" condition. In an ordinary queue, once the rear pointer reaches the last index of the array, no more elements can be inserted even if there is empty space at the beginning of the array (vacated by previous deletions). Therefore the primary advantages of using a circular queue over a regular one is:

- **Memory Efficiency:** A circular queue connects the last position back to the first position, creating a circle. This allows us to reuse the empty slots vacated by the front pointer.
- **No Shifting Required:** Unlike some linear queue implementations that shift all elements forward after a deletion, a circular queue manages space simply by wrapping indices using the modulo operator: $(\text{index} + 1) \% \text{SIZE}$.

C program implementing circular queue:

```
#include <stdio.h>
#include <stdlib.h>

#define SIZE 5

int items[SIZE];
int front = -1, rear = -1;

// Check if the queue is full
int isFull() {
    if ((front == rear + 1) || (front == 0 && rear == SIZE - 1)) return 1;
    return 0;
}

// Check if the queue is empty
int isEmpty() {
    if (front == -1) return 1;
```

```

    return 0;
}

// (i) Insert operation
void insert(int element) {
    if (isFull()) {
        printf("\n Queue is full!! \n");
    } else {
        if (front == -1) front = 0;
        rear = (rear + 1) % SIZE;
        items[rear] = element;
        printf("\n Inserted -> %d", element);
    }
}

// (ii) Delete operation
int delete() {
    int element;
    if (isEmpty()) {
        printf("\n Queue is empty !! \n");
        return (-1);
    } else {
        element = items[front];
        if (front == rear) {
            front = -1;
            rear = -1;
        } else {
            front = (front + 1) % SIZE;
        }
        printf("\n Deleted element -> %d \n", element);
        return (element);
    }
}

// (iii) Display operation
void display() {
    int i;
    if (isEmpty())
        printf(" \n Empty Queue\n");
    else {
        printf("\n Items -> ");
        for (i = front; i != rear; i = (i + 1) % SIZE) {
            printf("%d ", items[i]);
        }
    }
}

```

```

        printf("%d ", items[i]);
        printf("\n Front index -> %d \n Rear index -> %d \n", front, rear);
    }
}

int main() {
    int choice, val;

    while (1) {
        printf("\n--- Circular Queue Menu ---");
        printf("\n1. Insert\n2. Delete\n3. Display\n4. Exit\nEnter choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter value to insert: ");
                scanf("%d", &val);
                insert(val);
                break;
            case 2:
                delete();
                break;
            case 3:
                display();
                break;
            case 4:
                exit(0);
            default:
                printf("\nInvalid Choice!\n");
        }
    }
    return 0;
}

```

2.

Applications of a Stack

A Stack is a linear data structure that follows the LIFO (Last-In, First-Out) principle. Because of this behavior, it is indispensable for any process that needs to "remember" a previous state and return to it later. The key uses of a stack are:

- **Expression Handling:** Used for converting infix expressions to postfix/prefix and evaluating them.
- **Function Calls (Recursion):** When a function is called, the system pushes the return address and local variables onto the Call Stack. When the function finishes, it pops them to return control.
- **Undo/Redo Mechanisms:** In text editors, every action is pushed onto a stack. "Undo" simply pops the last action.
- **Backtracking:** Used in algorithms like Depth-First Search (DFS) or solving mazes, where you need to backtrack to the last decision point.
- **Syntax Parsing:** Compilers use stacks to check for balanced parentheses and tags (like in HTML/XML).

Algorithm: Postfix Expression Evaluation: To evaluate a postfix expression, we scan the expression from left to right and use a stack to store operands.

1. Initialize an empty stack to store the operands.
2. Scan the expression one symbol at a time from left to right.
3. If the symbol is an operand, push it onto the stack.
4. If the symbol is an operator:
 - a. Pop the top element
 - b. Pop the next top element
 - c. Perform the operation: $\text{Result} = \text{Op1 (operator) Op2}$
 - d. Push the result back onto the stack
5. For a valid expression, after the entire expression is scanned the stack will contain only one value, the final result.

Evaluating $ABC - D * + E \$ F +$

Symbol	Action	Stack (Bottom to Top)	Calculation / Result
A	Push	A	
B	Push	A,B	
C	Push	A,B,C	
-	Pop 2, Calc, Push	A, (B - C)	Op2 = C, Op1 = B Res = B - C
D	Push	A, (B - C), D	
*	Pop 2, Calc, Push	A, ((B - C) * D)	Op2 = D, Op1 = (B-C), Res = (B-C)*D
+	Pop 2, Calc, Push	(A + ((B-C) * D))	Op2 = (B-C)*D, Op1 = A, Res = (A + ((B-C)*D))
E	Push	(A+((B-C)*D)), E	
\$	Pop 2, Calc, Push	((A+((B-C)*D))\$E)	Op2 = E, Op1 = (A + ((B-C)*D)), Res = ((A+((B-C)*D))\$E)
F	Push	((A+((B-C)*D))\$E), F	
+	Pop 2, Calc, Push	((A+((B-C)*D))\$E)+F	Final Expression Result

3.

A. Breadth-First Search (BFS):

BFS explores the graph layer by layer. It visits all neighbors at the current distance before moving to nodes further away.

- **Mechanism:** Uses a Queue (First-In, First-Out).
- **Best For:** Finding the shortest path in an unweighted graph.

Example

Imagine a social network where you want to find "friends of friends."

Graph Structure:

- A is connected to B and C.
- B is connected to D.
- C is connected to E.

Traversal starting at A:

1. Visit A: Add neighbors B and C to the queue.
2. Visit B: (First in queue). Add neighbor D to the queue.
3. Visit C: (Next in queue). Add neighbor E to the queue.
4. Visit D: (Next in queue).
5. Visit E: (Last in queue).

BFS Order: $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E$

B. Depth-First Search (DFS)

DFS explores as deeply as possible along each branch before backtracking. It follows one path to its end before trying another.

- **Mechanism:** Uses a Stack (Last-In, First-Out) or Recursion.
- **Best For:** Solving puzzles (like mazes), cycle detection, and topological sorting.

Example

Imagine navigating a maze: you pick a path and walk until you hit a wall, then turn back to the last intersection.

Graph Structure (Same as above):

- A is connected to B and C.
- B is connected to D.
- C is connected to E.

Traversal starting at A:

1. Visit A: Move to first neighbor, B.
2. Visit B: Move to its neighbor, D.
3. Visit D: No more neighbors. Backtrack to B, then backtrack to A.
4. Visit C: Move to its neighbor, E.
5. Visit E: No more neighbors.

DFS Order: $A \rightarrow B \rightarrow D \rightarrow C \rightarrow E$

4.

A. Insertion Sort

Insertion Sort works by building a sorted section at the beginning of the array. It takes the next unsorted element and inserts it into its correct position among those already sorted.

The Algorithm

1. Start from the second element (index 1).
2. Compare the current element (key) with its predecessor.
3. If the key is smaller than its predecessor, compare it to the elements before.
4. Shift the greater elements one position up to make space for the swapped element.
5. Insert the key into its correct position.
6. Repeat for all elements until the array is sorted.

Example: [12, 11, 13, 5, 6]

Pass	State	Action
Initial	[12, 11, 13, 5, 6]	Start at 11
Pass 1	[11, 12, 13, 5, 6]	11 < 12, so 11 moves to the front.
Pass 2	[11, 12, 13, 5, 6]	13 > 12, stays put.
Pass 3	[5, 11, 12, 13, 6]	5 is smaller than all, shifts everything and inserts at index 0.
Pass 4	[5, 6, 11, 12, 13]	6 moves past 13, 12, 11 to sit after 5.

B. Selection Sort:

Selection Sort maintains two sub-arrays: one already sorted and one remaining unsorted. In every iteration, the minimum element from the unsorted sub-array is selected and moved to the end of the sorted sub-array.

The Algorithm

1. Initialize minIndex to the first position.
2. Search the remaining unsorted array to find the smallest element.
3. Swap the found minimum element with the element at the minIndex.
4. Increment minIndex to the next position.
5. Repeat until the entire array is sorted.

Example: [64, 25, 12, 22, 11]

Pass	Smallest Found	Swap Action	Resulting Array
1	11	Swap 64 and 11	[11, 25, 12, 22, 64]
2	12	Swap 25 and 12	[11, 12, 25, 22, 64]
3	22	Swap 25 and 22	[11, 12, 22, 25, 64]
4	25	Already in place	[11, 12, 22, 25, 64]